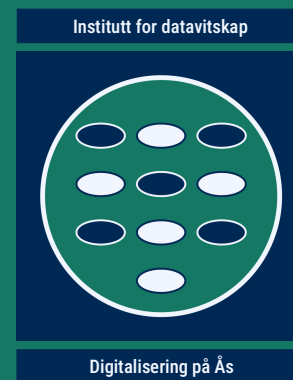
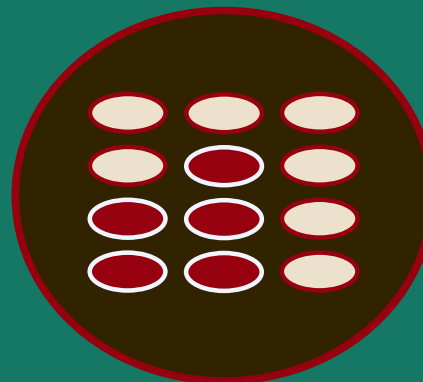


Noregs miljø- og
biovitenskaplege
universitet



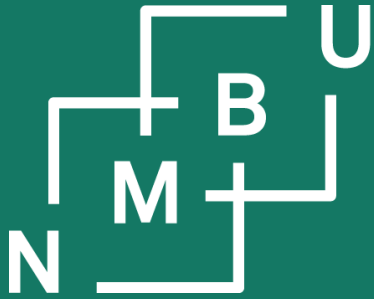
INF230

Datahandtering og -analyse

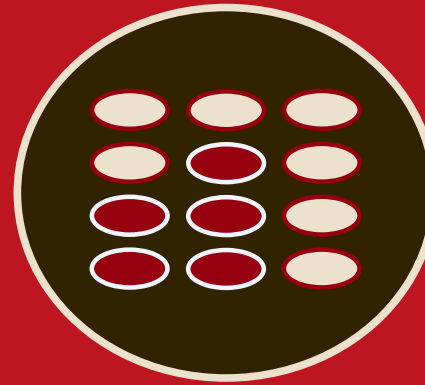
1 Ontologiar

- 1.1 Den semantiske veven
- 1.2 Vevontologispråket OWL
- 1.3 Terminologiutvikling

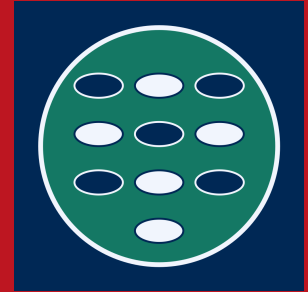
- 1.4 Ontologiutvikling**
- 1.5 DOLCE, EMMO og BFO
- 1.6 Semantisk heterogenitet



Noregs miljø- og
biovitenskaplege
universitet



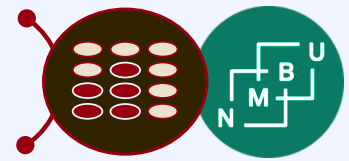
Institutt for datavitenskap



Digitalisering på Ås

1 Ontologiar

1.4 Ontologiutvikling



Termliste

nn brukarhistorie *f.*

en user story

nn domeneontologi *m.*

en domain ontology

nn epikk *m.*

en epic

nn kompetansespørsmål *n.*

en competency question

nn kunnskapsdomene *n.*

en domain of knowledge

nn persona *f.*

en persona

nn reifikasjon *m.*

en reification

nn smidig kravanalyse *m.*

en agile requirements analysis

Botn-opp-ontologiutvikling¹

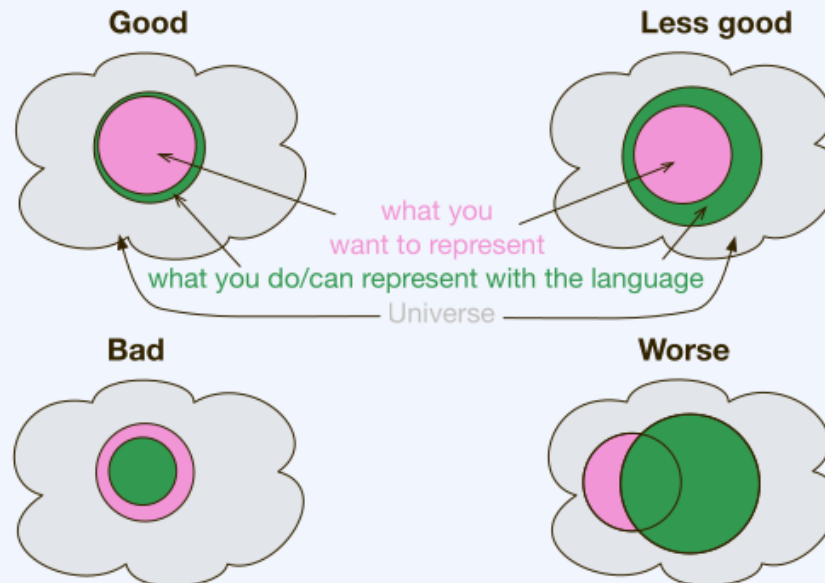
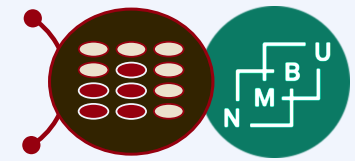
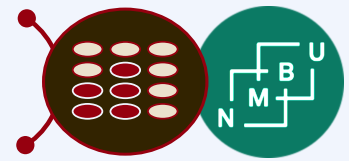


Figure 1.9: Good, less good, bad, and even worse ontologies. The pink circle denotes the subject domain that we want to represent in the ontology (say, African Wildlife) and the green circle denotes what is represented in the ontology (say, the AWO).

¹C.M. Keet, avsnitt 1.3.3: Good, not so good, and bad ontologies



Botn-opp-ontologiutvikling¹

- A ‘good’ ontology may contain the following three axioms:

1. Each lion has exactly 4 legs as parts.
2. Each lion is a carnivore.
3. Each lion eats at least some impala.

For our example, let us assume this is almost all there is to represent. We could distinguish between front and rear legs, and so it is not a 100% match, but it is close.

- A ‘less good’ ontology is not wrong in statements, but admits more models than intended. For instance:

- 1*. Each lion has at least some legs as parts.
2. Each lion is a carnivore.
3. Each lion eats at least some impala.

While the statements are technically not wrong, unlike the ‘good’ ontology, this one permits individual lions with 2, 5, or even 100 legs, which evidently is false. That is, this ontology is less precise compared to the previous one.

¹C.M. Keet, avsnitt 1.3.3: Good, not so good, and bad ontologies

Botn-opp-ontologiutvikling¹

- A 'bad' ontology contains less than what it is supposed to contain. For instance, it asserts only
 3. Each lion eats at least some impala.out of the three axioms that are supposed to be represented in the ontology.
- A 'worse' ontology contains out-of-domain knowledge, e.g.:
 3. Each lion eats at least some impala.
 4. Each painting depicts at least some bear.It misses axioms 1 and 2, and even if one were to bend over backwards and consider paintings of African wildlife animals also within scope: there are no bears in Africa.

¹C.M. Keet, avsnitt 1.3.3: Good, not so good, and bad ontologies

Ontology requirements from scenario descriptions

The ontology (TBox) contains the “universals” (categories) used in the knowledge graph (ABox) to describe a state of affairs. So one way to design an ontology is to look at how we would write down information on a state of affairs.

But we need to write it down as triples: **Subject**, **predicate**, **object**.

So for example “today it is 30°C in Saloniki” ...

Ontology requirements from scenario descriptions

The ontology (TBox) contains the “universals” (categories) used in the knowledge graph (ABox) to describe a state of affairs. So one way to design an ontology is to look at how we would write down information on a state of affairs.

But we need to write it down as triples: **Subject**, **predicate**, **object**.

So for example “today it is 30°C in Saloniki” could be rewritten as:

“ x_0 is a temperature measurement. x_0 has the location Saloniki. x_0 has the measurement value 30°C. x_0 has the measurement date 2025-07-05.”

Ontology requirements from scenario descriptions

The ontology (TBox) contains the “universals” (categories) used in the knowledge graph (ABox) to describe a state of affairs. So one way to design an ontology is to look at how we would write down information on a state of affairs.

But we need to write it down as triples: **Subject**, **predicate**, **object**.

So for example “today it is 30°C in Saloniki” could be rewritten as:

“ x_0 is a temperature measurement. x_0 has the location Saloniki. x_0 has the measurement value 30°C. x_0 has the measurement date 2025-07-05.”

We can continue e.g.: “Saloniki is a city. City is a subclass of location. 30°C is a temperature value. Temperature value is a subclass of quantity value. Etc.”

Exercise: Describe the state of affairs¹

Make some
selected
statements about
the conditions
described here.

Only use **simple sentences** of the form “**subject, predicate, object.**”

¹C. Zisis *et al.*, *Materials Today: Proceedings* **19(1): 65–72, 2019.**

These OPV panels of native area 70 x 25cm with 2.1% efficiency and transparency in the visible spectrum transmittance [20] were integrated by applying double-sided adhesive tape on the rooftop of a 24 m² Mediterranean Greenhouse surface covering the 22% of its total area. The OPV panels were mounted on the interior surface of the Mediterranean greenhouse to be protected from rain and dust. The plastic surface of the Mediterranean greenhouse was a Low Density Polyethylene (LDPE), enriched with compounds that absorb part of ultraviolet radiation and is located in the region of Thermi, Thessaloniki, Greece (Fig. 1).



Fig.1. Mediterranean greenhouse integrated with semi-transparent fully printed OPV modules based on P3HT:PCBM photoactive layer.

In order to evaluate the impact of OPV shading on plant growth and behavior, pepper plants (*Capsicum annuum*) inside pots were placed to the interior and exterior of the experimental Mediterranean greenhouse. The examination period lasted 154 days for a whole cultivation period between June and November. During this season, the average duration of daily sunshine was approximately ten hours. The pepper plants placed in the interior of the greenhouse were divided into two groups: the “OPV-Shaded Plants” (Fig. 2a) and the “Control Plants” (Fig. 2b). At the exterior of the greenhouse placed another group of plants referred as “Outdoor Plants” (Fig. 2c), which were exposed completely to the approaching solar radiation.

Reifikasjon av fleirplasspredikat

They don't want to just say that Shakespeare wrote *Hamlet*, but they want to qualify this statement and say that Shakespeare wrote *Hamlet* in 1604 or that Wikipedia states that Shakespeare wrote *Hamlet* in 1604. In general, these are cases in which it is, or at least seems, desirable to make a statement about another statement. This process is called *reification*. Reification is not a problem specific to Semantic Web modeling; the same issue arises in other data modeling contexts like relational databases and object systems. In fact, one approach to reification in the Semantic Web is to simply borrow the standard solution that is commonly used in relational database schemas

Subject	Predicate	Object
bio:n1	bio:author	lit:Shakespeare
bio:n1	bio:title	"Hamlet"
bio:n1	bio:publicationDate	1601

Allemang *et al.*, avsnitt 3.6: Higher-order relationships

Reifikasjon av fleirplasspredikat

This approach works well for examples like *Shakespeare wrote Hamlet in 1601*, in which we want to express more information about some event or statement. It doesn't work so well in cases like *Wikipedia says Shakespeare wrote Hamlet*, in which we are expressing information about the statement itself, *Shakespeare wrote Hamlet*. This kind of metadata about statements often takes the form of provenance (information about the source of a statement, as in this example), likelihood (expressed in some quantitative form like probability, such as *It is 90 percent probable that Shakespeare wrote Hamlet*), context (specific information about a project setting in which a statement holds, such as *Kenneth Branagh played Hamlet in the movie*), or time frame (*Hamlet plays on Broadway January 11 through March 12*). In such cases, it is useful to explicitly make a statement about a statement. This process, called *explicit reification*, is supported by the W3C RDF standard with three resources called `rdf:subject`, `rdf:predicate`, and `rdf:object`.

Allemang et al., avsnitt 3.6: Higher-order relationships

Ontology requirements from competency questions

Competency questions are questions that [somebody formulating requirements] wants a data infrastructure to be able to answer competently. As a requirements analysis method, it was established by Grüninger & Fox (1995).¹

For DigiPass CSA, we gathered 39 competency questions, including e.g.:

- “What hazardous substances does the given prepainted metal product contain?”
- “What is the climate and environmental impact of the prepainted metal product during the life cycle of the building?”
- “Identify a minimum value for the hourly wage such that nobody involved in the value chain for the product has been paid less.”
- *Etc. etc.*, they are all listed in DigiPass CSA deliverable D6.1.

¹M. Grüninger, M.S. Fox, in A. Rolstadås (ed.), *Benchmarking: Theory and Practice*, **1995**.

Briefly on agile requirements analysis¹

Main points of the approach include:¹

- Requirements analysis based on an ongoing dialogue with people.
- Diversity in the sort of people that might have requirements.
 - It is not always possible to actually interview and talk to representatives of all groups – but that is the ideal way of doing it.
- General good practice in formulating objectives: They must be SMART. Specially, there must be a clear criterion to see whether they are met.
- Suitable for agile style of development; focus less on static specifications and more on what something is needed for – maybe a new, better way can be found.
- “Black cat or white cat, it does not matter. But it has to catch the mice.”

¹M. Cohn, *User Stories Applied for Agile Software Development*, Boston: Addison-Wesley (ISBN 978-0-321-20568-1), **2004**

Briefly on agile requirements analysis¹

User story: “Describes functionality that will be valuable” to someone (Cohn). It has a format like “as a <persona>, I want to do <action> to achieve <aim>.”

- Good user stories are *independent, negotiable, valuable, estimable, small, and testable* (INVEST).

Epic: A coarse-grained objective. One way to distinguish epics and user stories is to look at the time that will be required to create the functionality.

- ISO 6515:2018 defines an epic as a “major collection of related feature sets broken down into individual features or user stories and implemented in parts over a longer period of time”.

Persona: “An imaginary representation of a user role” (Cohn). It is good practice to talk to actual people and take down their stories, but still present them in an aggregated and anonymized way.

¹M. Cohn, *User Stories Applied for Agile Software Development*, Boston: Addison-Wesley (ISBN 978-0-321-20568-1), 2004

From competency questions to SPARQL

This gives us a more precise understanding of competency questions.

A competency question is a requirement for designing a system of ontologies. It is formulated in natural human language.

The requirement is met if the ontologies can be used to express the question as a SPARQL query.

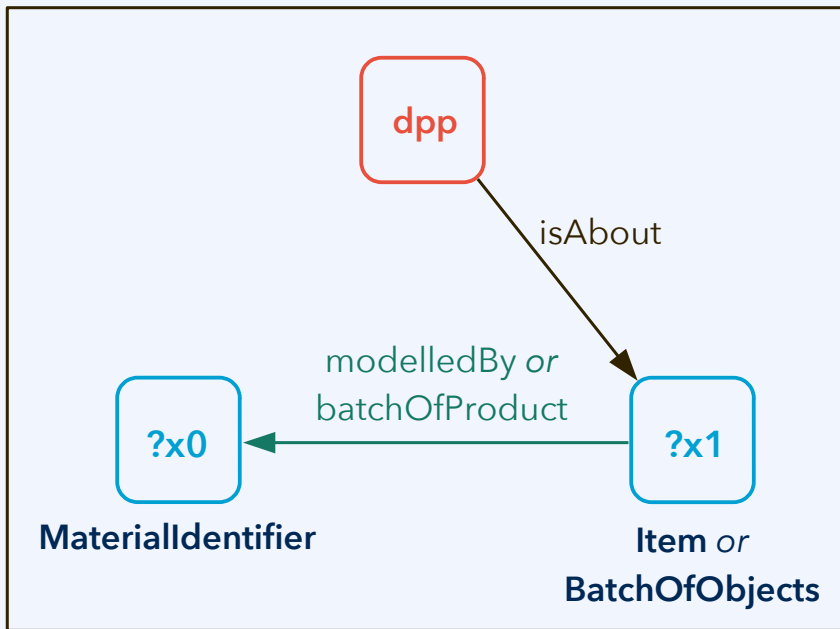
Example question from the Digi-Pass CSA requirements analysis:

“For a DMPP instance: Identify the material (product), and identify the batch or delivery unit.”

(Read as: “What is the ... ?”)

Competency questions to SPARQL

pattern



Example question from the Digi-Pass CSA requirements analysis:

“For a DMPP instance **dpp**: Identify material (product), and identify the batch or delivery unit.”

```
SELECT DISTINCT ?x0 ?x1 WHERE
{
  dpp mso-em:isAbout ?x1.
  {?x1 a ceon-prod:Item} UNION {?x1 a ceon-prod:BatchOfObjects}.
  {?x1 ceon-prod:modelledBy ?x0} UNION {?x1 ceon-prod:batchOfProduct ?x0}.
  ?x0 a pims-ii:MaterialIdentifier.
}
```

Competency questions to SPARQL

sd1: “For a DMPP instance: Identify the material (product), and identify the batch or delivery unit.”

Concepts:

- For a unit of product: `ceon-prod:Item`
- For a batch of product: `ceon-prod:BatchOfObjects`
- For identifying a kind of material: `pims-ii:MaterialIdentifier`

Relations:

- For the relationship between the given DMPP and the unit of product: `mso-em:isAbout`.
- For the relationship between the given DMPP and the batch of product: `mso-em:isAbout`.
- For the relationship between the unit of product and the kind of product: `ceon-prod:modelledBy`.
- For the relationship between the batch and the kind of product: `ceon-prod:batchOfProduct`.

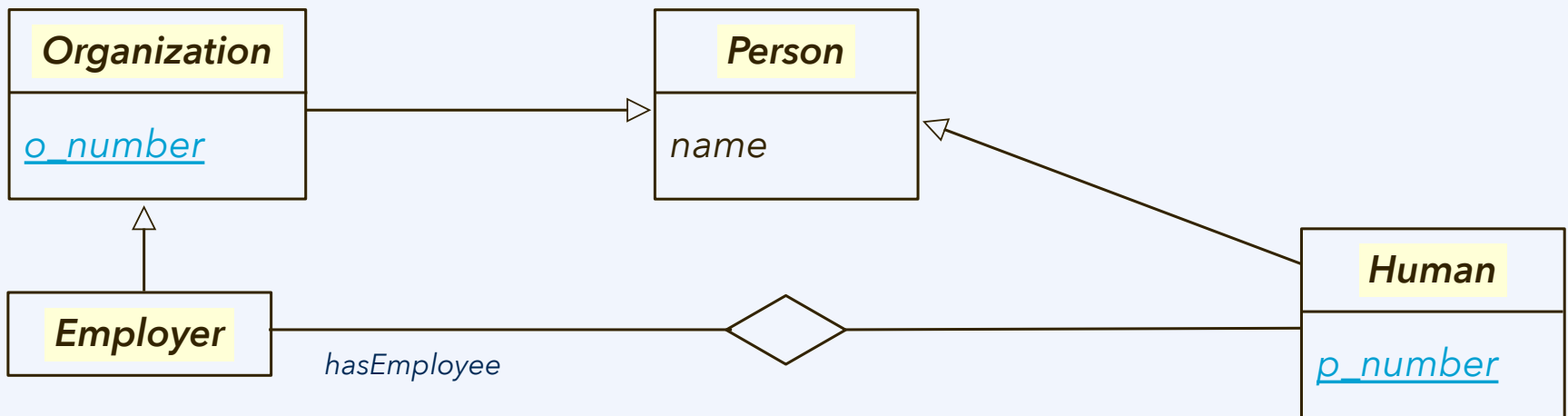
```
SELECT DISTINCT ?x0 ?x1 WHERE
{
  dpp mso-em:isAbout ?x1.
  {?x1 a ceon-prod:Item} UNION {?x1 a ceon-prod:BatchOfObjects}.
  {?x1 ceon-prod:modelledBy ?x0} UNION {?x1 ceon-prod:batchOfProduct ?x0}.
  ?x0 a pims-ii:MaterialIdentifier.
}
```

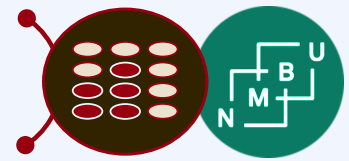
Problem 1.5 from the problem list

1.5 Competency question method

Extend to cover the following CQ: "What banks does some given person have bank accounts at?"

Prove that this was successfully done and that the CQ is indeed covered.





Oops! Evaluering av ontologidesign

Vi kan vurdere og dobbeltsjekke våre ontologiar ved bruk av **Oops!**¹
Oops! gjev tilbakemeldingar til ontologistruktur og -implementering.

- **Critical** 🚫 : It is crucial to correct the pitfall. Otherwise, it could affect the ontology consistency, reasoning, applicability, etc.
- **Important** ⚠️ : Though not critical for ontology function, it is important to correct this type of pitfall.
- **Minor** 🟡 : It is not really a problem, but by correcting it we will make the ontology nicer.

[\[Expand All\]](#) | [\[Collapse All\]](#)

Results for P11: Missing domain or range in properties.	195 cases Important ⚠️
Results for P13: Inverse relationships not explicitly declared.	45 cases Minor 🟡
Results for P36: URI contains file extension.	ontology* Minor 🟡
SUGGESTION: symmetric or transitive object properties.	5 cases

resultat frå
Oops!

Eksempel:

Har vi oppgjeve domene og kodomene for alle våre forholdstypar?

Definierer ontologien eit namnerom som instansierer owl:Ontology?

¹M. Poveda et al., *Int. J. Semant. Web Inform. Sys.* **10**(2): 7–34, doi:10.4018/ijswis.2014040102, **2014**.

Foops! Vurdering kor vidt ontologien er FAIR

Vi kan vurdere og dobbeltsjekke våre ontologiar ved bruk av **Oops!**¹ og **Foops!**²
Foops! sjekker om ontologien har blitt publisert på ein FAIR måte.

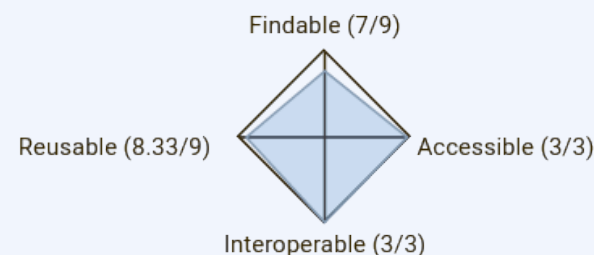
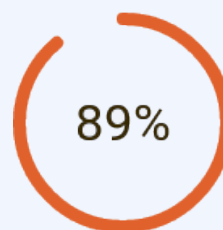
- **Critical** 🚫 : It is crucial to correct the pitfall. Otherwise, it could affect the ontology consistency, reasoning, applicability, etc.
- **Important** ⚠️ : Though not critical for ontology function, it is important to correct this type of pitfall.
- **Minor** 🟡 : It is not really a problem, but by correcting it we will make the ontology nicer.

[Expand All] | [Collapse All]

Results for P11: Missing domain or range in properties.	195 cases Important ⚠️
Results for P13: Inverse relationships not explicitly declared.	45 cases Minor 🟡
Results for P36: URI contains file extension.	ontology* Minor 🟡
SUGGESTION: symmetric or transitive object properties.	5 cases

resultat frå
Oops!

resultat frå
Foops!



¹M. Poveda et al., *Int. J. Semant. Web Inform. Sys.* **10**(2): 7–34, doi:10.4018/ijswis.2014040102, **2014**.

²D. Garijo et al., *Proc. ISWC 2021 Posters/Demos/Industry*, p. 321, **2021**.

For semantic interoperability: Reuse ontologies

If you design your own ontology, where possible, state how your concepts are the same as, or similar to, some concept in an ontology that already exist.

You can also avoid designing an ontology altogether if all the concepts you need can be brought together by combining a few pre-existing ontologies.

DigiPass CSA examples:

- The passport can be for a “unit of product” or for a “batch of product”. For these concepts, we can reuse the CEON ontology system (OntoDESIDE).
- When we want to say from when to when a process went, we can use “duration” from schema.org, which itself uses ISO 8601.

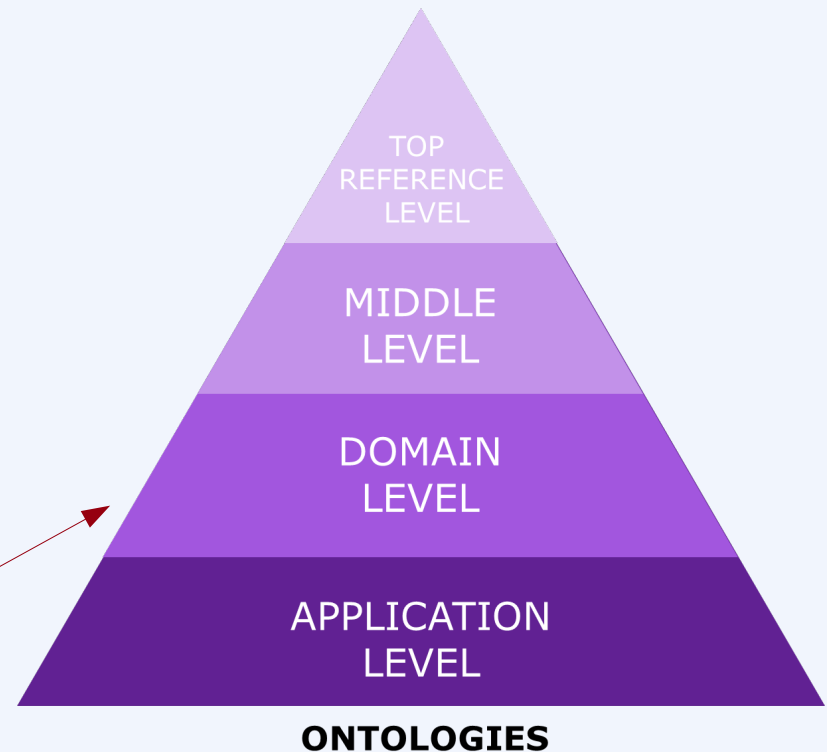
In one of our project deliverables, we have identified ten domain-level ontologies that we can reuse for concepts that we then don't need to define.

For semantic interoperability: Reuse ontologies

Semantics architectures often follow a hierarchy such as shown to the right.

Ontologies that are more specific, with a more narrow scope, refer to those that are more general or at the same level as themselves.

We refer to these as "domain ontologies."



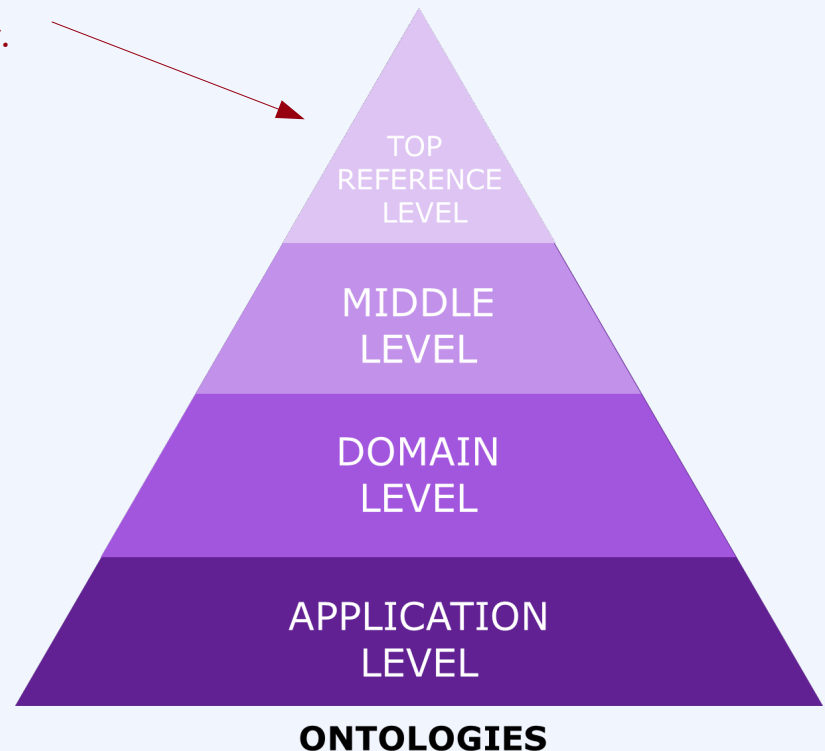
For semantic interoperability: Reuse ontologies

These here are called “top-level” but also “foundational” ontologies.

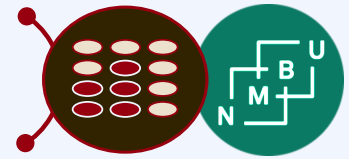
Semantics architectures often follow a hierarchy such as shown to the right.

Ontologies that are more specific, with a more narrow scope, refer to those that are more general or at the same level as themselves.

The most abstract level is that of the **foundational ontologies**, which we will look at in the next lecture.



Glossar



nn **kompetansespørsmål** *n.*

krav til ein kunnskapsrepresentasjonsformalisme, formulert som eit spørsmål i vanleg menneskespråk, som kunnskapsrepresentasjonsformalismen skal kunna uttrykkje formelt

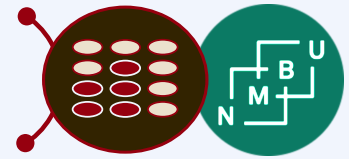
nn **reifikasjon** *m.*

transformasjon frå éin semantisk formalisme, der eit visst uttrykk opptrer som predikat, verb, forholdstype, forhold e.l., til ein annan, der det umiddelbart tilsvarar ei klasse, eit substantiv, nominaluttrykk, objekt, individ, eller liknande

nn **smidig kravanalyse** *m.*

kravanalyse der elementære krav blir formulert som brukarhistorier og gruppert i epikkar

Glossar



nn **kompetansespørsmål** *n.*; en **competency question**

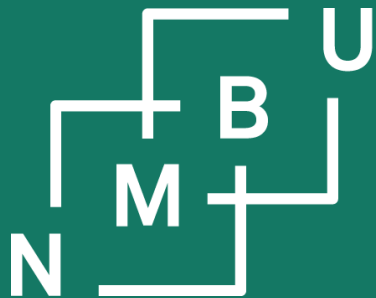
krav til ein kunnskapsrepresentasjonsformalisme, formulert som eit spørsmål i vanleg menneskespråk, som kunnskapsrepresentasjonsformalismen skal kunna uttrykkje formelt

nn **reifikasjon** *m.*; en **reification**

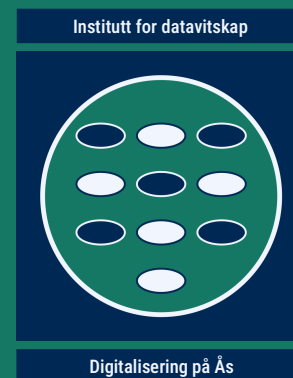
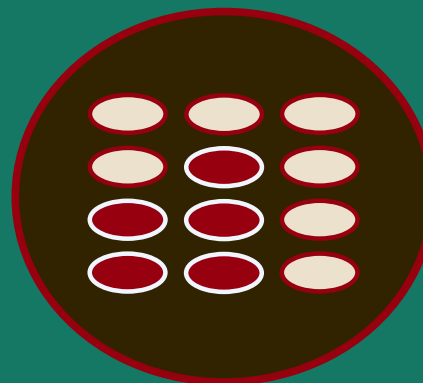
transformasjon frå éin semantisk formalisme, der eit visst uttrykk opptrer som predikat, verb, forholdstype, forhold e.l., til ein annan, der det umiddelbart tilsvarar ei klasse, eit substantiv, nominaluttrykk, objekt, individ, eller liknande

nn **smidig kravanalyse** *m.*; en **agile requirements analysis**

kravanalyse der elementære krav blir formulert som brukarhistorier og gruppert i epikkar



Noregs miljø- og
biovitenskaplege
universitet



INF230

Datahandtering og -analyse

1 Ontologiar

- 1.1 Den semantiske veven
- 1.2 Vevontologispråket OWL
- 1.3 Terminologiutvikling

- 1.4 Ontologiutvikling**
- 1.5 DOLCE, EMMO og BFO
- 1.6 Semantisk heterogenitet